PASTA Threat Model — Cliente A

> **Anonymization note.** This report documents real work performed under authorization from the owner. The client's name and infrastructure identifiers were replaced with placeholders (`Cliente A` , `cliente-a`) before publication. Belona Cybersecurity does not publish identifiable client data without written authorization.

Stages 3 to 7 (Stages 1-2, Objectives and Scope, documented previously)

PASTA = Process for Attack Simulation and Threat Analysis. A risk-centric threat modeling methodology in 7 stages. This document covers stages 3-7 applied to the system (web + Firebase).

Stage 3 — Application Decomposition (ADA)

Components

Component	Technology	Role
Public frontend	HTML/CSS/JS (Firebase Hosting)	Site + contact form
Admin panel	admin.html	Management (guides, blog, inquiries, cases, users, logs)
Authentication	Firebase Auth + Identity Platform	Login + TOTP MFA
Database	Firestore (`nam5`)	guias, servicios, posts, consultas, procesos_insolvencia, auditoria
Files	Firebase Storage	Case documents
Backend	Cloud Functions (Node 24)	enviarConsulta, admin*, auditarSituacion

Actors / Roles

- **Anonymous public** (site visitors; submit inquiries)
- **editor** (edits blog), **abogado/lawyer** (services/inquiries/cases), **admin** (everything + users)
- **External attacker**, **malicious insider** (low-privilege role)

Assets to protect

- Client personal data (inquiries, case files) — **sensitive, Law 1581 / Colombian data protection**
- Legal documents (Storage)
- Credentials and secrets
- Site integrity and availability

Trust boundaries

1. Browser ↔ Firebase (Internet). 2. Client ↔ Firestore/Storage rules (server-side).
2. Client ↔ Cloud Functions (App Check + auth). 4. Repo/CI ↔ deployment.

Entry points

Contact form · Panel login · Cloud Function calls · Public content reads · CI/CD pipeline.

Stage 4 — Threat Analysis (TA)

Threat sources: opportunistic/targeted external attacker, bots, low-privilege insider, human error, supply chain.

Main threats (STRIDE + real context): 1. **Unauthorized data access** (Information Disclosure) 2. **Privilege escalation** (Elevation of Privilege) 3. **Injection / XSS** in public-origin data (Tampering) 4. **Credential theft / account takeover** (Spoofing) 5. **Code/secret leakage** (Information Disclosure — deployment chain) 6. **Form abuse / cost DoS** (Denial of Service) 7. **Tampering with served files** (defacement) 8. **Supply chain** (vulnerable dependencies)

Stage 5 — Vulnerability & Weakness Analysis (WVA)

Weakness	CWE/OWASP	Status
`.git` exposure + token (deploy config)	CWE-538 / OWASP A05	Found in pentest, fixed
Server code exposed in Hosting	CWE-540	Fixed
Missing security headers (clickjacking, MIME)	OWASP A05	Fixed
Vulnerable dependencies (functions)	OWASP A06	Mitigated (no high/critical) + Dependabot + CI audit
Inline scripts without strict CSP	OWASP A05	Accepted (low risk; CSP frame-ancestors set)
Public contact endpoint (abuse)	OWASP A04	Mitigated (App Check + server-side validation)
Stored XSS in inquiries/cases	OWASP A03	Mitigated (`esc()` escaping in the panel)

Stage 6 — Attack Modeling (AM)

Attack trees for top threats (and the control that cuts each path):

Attacker goal: read client data - Path 1: read Firestore directly without login → **BLOCKED** by rules (validated: permission-denied) - Path 2: log in as admin → requires credential **+ MFA** (mitigated) - Path 3: low-

privilege user (editor) reads inquiries → **BLOCKED** by rules (validated) - Path 4: download the repo via `.git` → was **OPEN**, now **CLOSED**

Goal: become admin (escalation) - Call `adminAsignarRol` as editor → **BLOCKED** (App Check + `exigirAdmin`, validated)

Goal: tamper with the site (defacement) - Alter served files → detected by **FIM** (validated with Telegram alert)

Goal: steal case documents - Access Storage without a role → **BLOCKED** (rules `storage/unauthorized`, validated)

Stage 7 — Risk & Impact Analysis (RIA)

Risk = Likelihood × Impact. High impact due to sensitive legal data (Law 1581).

Threat	Likelihood (with controls)	Impact	Residual risk
Unauthorized data access	Low	High	**Low**
Privilege escalation	Low	High	**Low**
Stored XSS	Low	Medium	**Low**
Credential theft	Low (MFA)	High	**Low-Medium**
Code/secret leakage	Low (fixed)	High	**Low**
Cost abuse / DoS	Medium	Low	**Low** (App Check + budget alert)
Defacement	Low	Medium	**Low** (FIM detects)
Vulnerable dependencies	Low	Medium	**Low** (Dependabot + CI audit)

Implemented countermeasures

Prevention: RBAC (admin/lawyer/editor), Firestore/Storage rules, App Check, MFA, server-side validation, anti-XSS escaping, security headers, secret management. Detection: FIM + audit Logs. Recovery: backups (PITR + daily + offsite 3-2-1) + runbooks. Compliance: consent with proof, Law 1581 policy with international transfer clause.

Recommendations (accepted risk / future)

1. Strict script CSP (refactor inline). 2. Field validation in rules (size/type).
2. Migrate GEMINI_KEY to Secret Manager. 4. Budget alert. 5. Re-run pentest after major changes.

Conclusion

The system presents **low residual risk** across the relevant threats, with controls in all three layers (prevention, detection, recovery) validated via authorized penetration testing. The model should be reviewed upon significant changes.
